SumVP: RDF Summary Vertical Partitioning for Accelerating Complex SPARQL Queries

Abstract—With fast proliferation of RDF data, it has become a very promising solution for distributed RDF management systems to store, distribute and query large amounts of data built on top of Hadoop ecosystem. How to efficiently handle and accelerate complex SPARQL queries over a large scale of RDF data has become challenging. In this paper, we propose a novel RDF summary-based framework to accelerate complex SPARQL queries by RDF summary-based vertical partitioning and execution plan optimization over Spark. First, a lossless graph summarization approach is presented to produce RDF summary graph, where hash table-based summary structure indexing is maintained. Second, an RDF summary-based vertical partitioning approach (SumVP) is proposed to partition the summary into binary tables (BTs), through which a relational join-based preprocessing is utilized to build pre-join ternary tables (TTs) by computing the possible join relations between BTs. Third, complex SPARQL queries are made based on the summary by a BT/TT statistics-based execution plan optimization, where we rewrite SPARQL queries into Spark SQL queries, reorder the query triples by BT/TT-based statistics, and obtain the results by hash table structure-based summary recovery. Last but not least, extensive experiments were made over four variants of the benchmark datasets LUBM and WatDiv in comparison with the state-of-the-art approaches, and the results show that our approach is very competitive against these approaches for accelerating complex SPARQL queries.

Index Terms—RDF Summary, Summary Vertical Partitioning, Ternary Table, Summary Structure Indexing, Query Rewriting.

1 Introduction

Resource Description Framework (RDF) represents data from diverse sources in a versatile manner as a set of triples of the form <subject, property, object>. A triple connects a pair of entities associated with named attribute values by a named relationship, and a set of triples can be regarded as a directed and labeled graph. RDF data has drastically expanded in the last decades. Typical RDF datasets are DBPedia [1], Bio2RDF [2], Knowledge Graphs [1, 2] and Linking Open Data (LOD) [3], including millions of and even billions of publicly available triples. SPARQL[3] is the standard query language over RDF data. With more and more data exposed in RDF format, we are also witnessing a compelling need to formulate more or less sophisticated queries over the RDF data. It is reported that many queries in some existing corpuses contain a large number of query pattern triples [4]. Typical examples of large queries are from DBpedia15 including 209 and 211 triples, and BioMed13 including 221 and 229 triples. The proliferation of RDF data and the increasingly complex queries pose challenging issues for large-scale RDF data management and efficient SPARQL query evaluation.

Nowadays, distributed approaches for large-scale and scalable distributed RDF data management have been intensively explored [5–9]. Distributed approaches for handling large-scale RDF data can be roughly divided into three categories: Standalone distributed RDF management systems, federation of centralized systems, and big data technology/cloud-based distributed RDF management systems. Standalone distributed RDF management systems deploy custom dedicated native stores, such as TriAD [10]. The use of these special stores somewhat possibly curbs the power of handling and storing RDF data from different types of sources. Federation of centralized systems integrate a number of centralized systems by coordinating a cluster of multiple nodes [11], such as SGDQ [12], Worq [13], Sparti [14] and GoFast [15]. They partition and distribute RDF data in different nodes of the cluster. An RDF query is decomposed into several subqueries each of which is executed on a single node. With the rapid expansion of big data and cloud computing technology in recent years [16–18], distributed RDF management systems built on top of Hadoop/Spark ecosystems [19–21] have illustrated promising potential in storing and handling large-scale RDF data, where distributed file system HDFS and Spark in-memory cluster computing are used for storing and processing RDF data, such as H2RDF+ [22], PigSPARQL [23], S2RDF [24], Sempala [25], SPARQLGX [26], SparkIlink [27, 28], Sparklify [13, 29], S3QLRDF [30], SparkRDF [31] and SHARD [32]. In this paper, we confine our work to big data technology-based distributed RDF management systems and explore how to accelerate complex SPARQL queries over large scale of RDF data and the Apache Spark platform.

Problem Statement. Efficient RDF data querying for large-scale RDF data on Spark is a challenging open problem due to that it is involved in some closely related aspects including efficient RDF data storage, partitioning, indexing and query plan optimization. First, existing

1. https://wiki.dbpedia.org/
2. https://bio2rdf.org/
3. http://www. w3. org/TR/rdf-sparql-query/

RDF data stores are mainly based on triples, property tables, graph model and binary tables. Triple-based RDF data storage scheme often needs a large number of cross joins for query execution. Property Table with multiple columns [33] is efficient for star-shaped queries due to diminishing the subject-subject joins in queries, but property tables are inefficient to handle other shapes of queries, large number of null values, and multi-valued attributes. Graph-based model [34] can be flexibly handled by dedicated graph processing APIs based on graph databases, but it is seldom efficiently applied to Spark/HDFS because graph queries over a large-scale graph mode often deal with high computational overhead for (sub-)graph isomorphism. Binary table [20] uses a single two-column table for each property in the RDF dataset and overcomes the limitation of null values, but binary table-based complex RDF query evaluation may need a large number of cross-table joins when querying multiple binary tables.

Second, the basic idea of RDF data Partitioning is to place the tuples that may be involved in the join into the same node for significantly reducing the cross-node/partitions joins. Typical vertical partitioning approaches over Hadoop/Spark, such as [13, 14, 18, 23, 29, 32], can hash some RDF element (e.g., subject, predicate or object) in a column-wise manner but they are only efficient to star query pattern [21]. Horizontal partitioning generally is to partition the RDF data into files by subject, but it seems to be not efficient for retrieval of suitable triples when the subject is a variable in the query pattern [7]. It is very desirable for complex RDF queries to achieve consistent querying performance over different shapes of queries including linear, star and snowflake, and more complex combinations based on them.

Third, indexing is adopted to accelerate data access and improve the query runtime performances for the data management systems. Existing typical indexing techniques are made by predicate, triple pattern and relational models, which generally deal with triple-permutation indexing [19, 22]. They often suffer from a severe storage overhead and are expensive to be maintained due to different semantics of RDF data [35].

At last, join operations for queries with many pattern triples will produce huge amount of intermediate results due to that SPARQL queries are join-intensive, which hinders querying performance improvement. Existing strategies [15, 36, 37] for distributed query optimization like pruning [15], predefined ontology [36] and heuristic query rewriting [37] either are based on non-HDFS systems or need prior knowledge. There are a few distributed query optimizations based on HDFS [21, 24, 26, 30], but they still suffer from a severe storage overhead and expensive distributed joins over tremendous intermediate results produced from its subqueries.

To accelerate complex SPARQL queries over Spark, we present a novel framework of RDF summary-based vertical partitioning and binary tables (BTs)/ternary tables (TTs) statistics-based execution plan optimization including relation pre-joins and query triple ordering. SPARQL queries over the original RDF data can be reduced to querying the RDF summary data for diminishing the input size of the query and the intermediate results. An RDF summary graph represents a compact informative representation by capturing coarse-grained structural features of the original RDF data. The produced summary instead of the raw RDF data is vertically partitioned to build BTs for each predicate. We further establish the possible relation pre-joins (TTs) between BTs to optimize execution plan based on the summary. The produced summary is utilized because 1) it is helpful in quickly access the raw graph data by means of the cliques of entities and relationships without traversing all entities and relationships in the raw graph. 2) since the size of the produced summary is much smaller than that of the raw RDF data, we can reduce the cost of query execution plan by BT/TT statistics-based relation pre-joins and query triple ordering. To the best of our knowledge, this is the first work to use RDF summary-based vertical partitioning, structural indexing and execution plan optimization for accelerating complex SPARQL queries on Spark. The main contributions of this paper are as follows.

- We present an RDF summary-based framework for accelerating complex SPARQL queries over large scale of RDF data, regardless of shapes of queries.
- A lossless graph summarization approach is presented to produce RDF summary graph where a hash table-based indexing structure is maintained for fast visiting information in the summary.
- An RDF summary-based vertical partitioning approach (SumVP) is proposed to build BTs of the summary, where a relational joins-based preprocessing is utilized to build TTs by computing the possible join relations between BTs for diminishing the input data of the query.
- We present a BT/TT statistics-based execution plan optimization by query triples reordering, based on which hash-based summary recovery can be made for filtering spurious joins from the intermediate results.
- Last but not least, extensive experiments were made, and the results show that our approach is very competitive to the state-of-the-art approaches over four variants of popular benchmark datasets LUBM and WatDiv.

This paper is organized as follows. Section 2 is the related work. In section 3, we briefly introduce the preliminaries and the overview of our framework. In section 4, we describe our lossless graph summarization-based data partitioning. In section 5, we make vertical partitions statistics-based execution plan optimization and summary recovery. Section 6 is about the experimental evaluation. Section 7 is the conclusion.

2 RELATED WORK

Data layouts play a crucial role in SPARQL queries over distributed RDF data management systems. For efficiency

reasons, triple-permutation of partial or all the columns is often adopted. Query evaluation is basically reduced to a serials of join operations based on these large tables [19, 22], which often suffers from expensive computational cost [35]. A vertical partitioning-based storage partitioning approach [20] was proposed to build a binary table for each predicate in the RDF data for reducing the redundant data storage. Much work has been made based on vertical partitioning of the RDF data, such as gSPARQL [23], SPARQLGX [26], SparkIlink [27, 28], Sparklify [13, 29], and SPARTI [14]. S2RDF [24] extends the work of [20] and presents an ExtVP (Extended Vertical Partitioning) approach, which uses a semi-join-based preprocessing to minimize the input of data and avoids dangling tuples (no any join with other tuples) for reducing unnecessary joins and memory cost. What is worth noting is that existing distributed approaches often translate SPARQL queries into specific query languages for scalable query evaluation. S2RDF translates SPARQL queries into the Spark SQL language[4]) handled by Spark SQL API for handling native data layout (e.g., Parquet [5]). Sempala [25] converts a SPARQL statement into SQL one and use the distributed RDF engine on the top of Apache Impala [38]. PigSPARQL [23] performs SPARQL queries by translating them into the scripts of Pig Latin language over Apache Pig [39].

Property Table (PT) is another common data layout scheme with multiple columns/predicates. PT has a significant advantage to star-like queries without any subject-subject joins for obtaining query results. However, the join operations over PTs will be expensive when queries contain the object-subject joins or the subject-object joins (e.g., linear queries). Typical property table-based approaches such as Sempala [25] and S3QLRDF [30]. Sempala uses PT tables to store the RDF data, where Parquet is adopted as the RDF data layout. S3QLRDF is based on Spark and presents a new data partitioning scheme called Property Table Partitioning (PTP), which divides property table into multiple sub-tables according to their predicates for minimizing the input size of data. S3QLRDF makes the relevant statistics about ExtVP tables for optimizing the order of pattern triples. There are also some hybrid data layout schemes for RDF data processing [21], which often use vertical partitioning and property tables to produce binary tables and property tables, but these methods are often efficient for star shape of queries. AdPart [5] partitions the original RDF triples by hashing the subjects of triples, and the triples with the same subject are stored in the same partition. The indexing in terms of predicate, predicate-subject and predicate-object is indexed for supporting different shapes of queries. However, due to that data partitioning and indexing are made based on the whole of raw RDF data, it still have to take much time to evaluate SPARQL queries over large-scale RDF data.

There are some summary-based RDF data partitioning approaches for RDF data queries such as TriAD [10] and SGDQ [12], where the data storage is limited to main memory capacity by the master-slaves architecture. In TriAD, summary graph is used for join-ahead pruning and diminishing expensive workload when SPARQL statements are executed. SGDQ uses graph summary for query matching and join-ahead query evaluation. We believe that a combination of memory and HDFS will be more efficient for large-scale RDF data processing and querying.

The current most relevant works to us are S2RDF [24], SPARQLGX [26], S3QLRDF [30], and CliqueSquare [40], where the techniques related to HDFS-based data storage, data vertical partitioning, table-based pre-joins and query optimization are exploited. Our work is different from the above approaches in terms of some features as follows. First, the raw RDF data is first processed by an aggregation-based lossless graph summarization to form a graph summary with the small-sized and compressed RDF data. The entire summary is hierarchically indexed by hashing the supernodes (the set of subjects and objects in the RDF triples). Vertical partitioning is further made to produce binary tables of the summary, each of which contains a significantly smaller number of tuples than that of the raw RDF data. Second, different from join-ahead pruning or semi-join operations during query execution, we use a relational join-based BT table preprocessing to build pre-join ternary tables (TT) from BT tables by computing the possible join relations between partitions of BT tables. The pre-joins based on TT tables can reduce the input data of SPARQL queries. Third, we use a pattern triples sorting and BT/TT statistics-based approach for optimizing complex SPARQL queries and evaluation by the hash-based summary recovery, which can further filter spurious joins from the intermediate results. Another advantage of our approach is that it provides improvements for all shapes and also for queries with a large diameter.

3 PRELIMINARIES AND OVERVIEW OF APPROACH

3.1 Preliminaries

RDF data can be regarded as a set of triple of the form <subject, predicate, object>. Let U, B and L be the set of URL resource identifier, blank nodes, and literals, respectively. The set T of triples for the RDF data can be represented as $T = (s, p, o) \mid s \in S, p \in P, o \in O$, where $S \subseteq U \cup B$, $P \subseteq U$ and $O \subseteq U \cup B \cup L$.

Definition 1. (**RDF Graph**) Let RDF graph $G = (V_G, E_G, P_G, \phi_G)$ be a directed and labeled graph, where $V_G \subseteq U \cup B \cup L$ is the set of all the subjects and objects in RDF triples, $E_G \subseteq V_G \times V_G$ is the set of directed edges,$P_G \subseteq P$ is the set of edge labels, and $\phi_G : E_G \rightarrow 2^{P_G}$ is a mapping function that assigns a subset of P_G to each edge $e \in E_G$.

Figure 1 (a) is an example of RDF triples, and Figure 1 (b) is the corresponding RDF graph.

A summary graph is a coarse-grained RDF data that preserves key characteristics in the original RDF data.

4. https://spark.apache.org/sql/
5. https://parquet.apache.org/

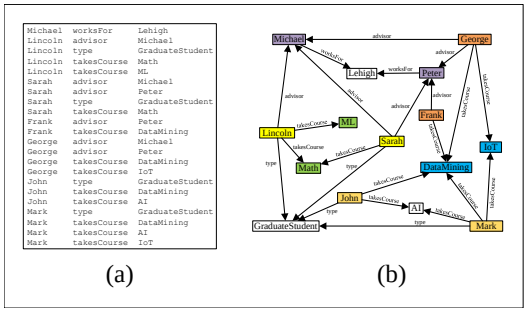

Fig. 1: An example of RDF data

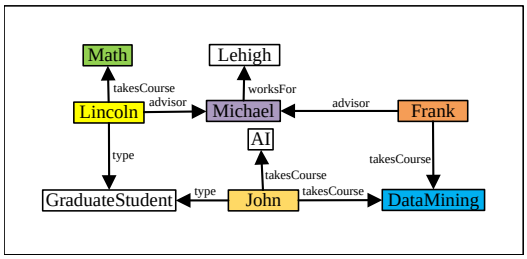

Fig. 2: An example of RDF summary graph

Definition 2. (**Summary Graph**) For an RDF graph $G = (V_G, E_G, P_G, \phi_G)$, a summary graph of V_G is defined as $S^G = (V_S^G, E_S^G, P_S^G, \phi_S)$, where V_S^G is the set of supernodes and $V_S^G = \varphi_i \mid \cup_{i=1}^{k} \varphi_i = V_G \wedge \forall 1 \leq i, j \leq k \wedge i \neq j, \varphi_i \cap \varphi_j = \emptyset$, where $k = |V_S|$ is a predefined number, and $\forall \varphi_i \in V_S^G (1 \leq i \leq k)$, we have $|\varphi_i| > 0$. E_S^G is the set of superedges and for any superedge $< \varphi, \varphi' > \in E_S^G$, it represents a full connections between φ and φ', i.e, $\forall u \in \varphi \ \forall v \in \varphi', (u, v) \in < \varphi, \varphi' >$. $P_S^G = P_G$ is the set of property labels in G. ϕ_G assigns each superedge $< \varphi, \varphi' > \in E_S^G$ a set of property labels, i.e., $\phi_S(\varphi, \varphi') = \bigcup_{(u,v) \in E_G \wedge (u,v) \in < \varphi, \varphi' >} \phi_G(u, v)$.

Figure 2 is a summary graph of the RDF data in Figure 1, where the original nodes with the same color will be aggregated into a supernode. For example, supernode *Lincoln* in figure 2 represents a set of original nodes *Lincoln* and *Sarah*. Note that the original RDF graph can be also regarded as a summary graph of its own without any node merge, where an original node is just a supernode.

A SPARQL query QT is a set of query pattern triples of the form $\langle s, p, o \rangle$,where s, p and o are either variables or bound elements in the RDF data. SPARQL query QT can be regarded as a directed and labeled graph.

Definition 3. (**SPARQL Query Graph**) A SPARQL query graph Q can be defined as $Q = (V_Q, V_{var}, E_Q, P_Q, \phi_Q)$, where $V_Q \subseteq V_G$ is the set of bound elements, V_{var} is the set of query variables, and $V_Q \cup V_{var} = \{s, o \mid \langle s, p, o \rangle \in QT\}$. $E_Q \subseteq (V_Q \cup V_{var}) \times (V_Q \cup V_{var})$, and a mapping function $\phi_Q : E_Q \rightarrow P_Q$ assigns each $e \in E_Q$ a property $p \in P_Q$.

Figure 3(a) is a SPARQL query and Figure 3(b) represents its query pattern graph.

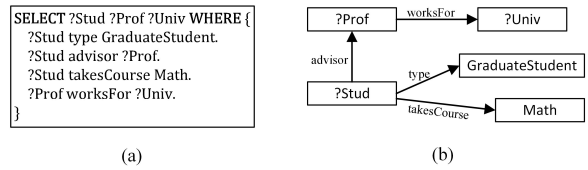

Fig. 3: An example of SPARQL Query

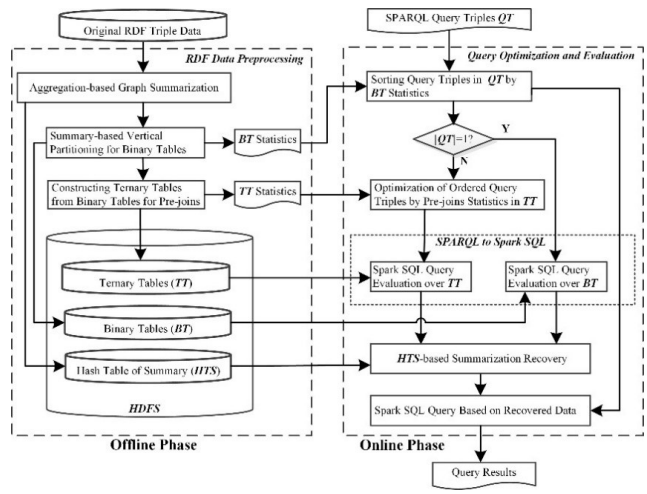

Fig. 4: Overview of Framework

3.2 Overview of Proposed Approach

The overall framework of our approach is shown in Figure 4, consisting of two stages including offline preprocessing of the RDF data, and SPARQL query optimization and evaluation. The former is about graph summarization and pre-joins based on BTs and TTs, and the latter is online to sort query pattern triples and optimize the order of query triples by pre-join and BT/TT statistics.

In the stage of RDF data preprocessing, the original RDF triples data is first pro-cessed by a node aggregation-based lossless graph summarization, and the produced summary is indexed by hashing supernodes (cliques of subjects/objects in the RDF data) in the summary for filtering spurious joins from intermediate results during query evaluation, stored in HDFS. Next, the summary is further vertically partitioned into BTs including two columns of subjects and objects. Each BT is built for each predicate and stores the rows of supernode pairs in terms of the predicate. Further, we construct TTs by computing the possible join relations with different join types between BTs of the summary. The BT/TT statistics are made for execution plan optimi-zation during query evaluation. This is discussed in section 4.

In the stage of query execution plan optimization and evaluation, we order query triples by BT statistics, where the query triple whose predicate pertaining to BT with smaller size should be placed as forward as possible for diminishing intermediate results. Next, we differentiate two kinds of situations. If the query is a single triple ($|QT|$=1), we only perform query evaluation over BTs.

Otherwise, for SPARQL que-ries with multiple query triples, which further needs to reorder of query triples by TTs statistics, and then perform query evaluation over TTs. During query evaluation, SPARQL queries will be rewritten into Spark SQL statements. Due to that the intermediate results produced by query evaluation over BTs and TTs may contain spuri-ous join relations, the structure indexing-based summary recovery will be used, and the final results will be obtained over the recovered data. This is discussed in section 5.

4 Summary based Vertical Partitioning

4.1 Lossless RDF Graph Summarization

In this paper, inspired by the classic distributed graph summarization in [35], we make an extension and further present a lossless RDF graph summarization method, where actual information in the raw RDF data will be preserved in the produced summary graph for summary-based recovery of the future. We tolerate to introduce spurious edges as less as possible for calculating full connections between two (super-)nodes and merging them, but do not remove any actual edge from the original graph. Alternatively, we maintain the corresponding set of actual edges belonging to each superedge by a hash-based indexing structure without storing spurious connections. As such, the indexing structure can facilitate RDF summary-based SPARQL queries by excluding spurious relational joins in intermediate results for fast recovering the actual query results from these intermediate results.

4.1.1 Calculation of Summarization Error

Graph summarization needs to determine if two supernodes are likely to be merged by calculating their merge error [35, 41]. The merging error of two nodes should be as small as possible. Here, we first define the similarity between two supernodes in equation 1.

$$\mathrm{sim}(\varphi, \varphi') = \frac{|N_\varphi \cap N_{\varphi'}|}{|N_\varphi \cup N_{\varphi'}|} \tag{1}$$

Where $N_\varphi = \{\varphi'' \in V_S^G \mid <\varphi, \varphi''> \in E_S^G \vee <\varphi'', \varphi> \in E_S^G\}$ representing the set of all the neighbors of φ.

If two supernodes $\varphi_i, \varphi_j \in V_S^G$ has been aggregated into a new supernode φ_m, then both φ_i and φ_i are no longer supernodes in V_S^G and therefore will be replaced with φ_m. We denote it as $\varphi_m = \pi(\varphi_i) = \pi(\varphi_j)$ and $\varphi_m = \varphi_i \cup \varphi'_j$, where $\pi(\varphi)$ refers to the supernode in which φ resides. Ideally, if there exist full connections between φ_i and φ_j, i.e., $\forall u \in \varphi \forall v \in \varphi'.(u, v) \in E_G$, then merging φ_i and φ_j will not introduce merge error. But in real-world graphs, there seldom exist full connections between two supernodes, which will inevitably introduce merge error. We call it inner merge error (*IME*). In additions, there possibly exists the error introduced by merging φ_m and its other neighbors, which is called adjacent merge error (*AME*).

Definition 4. (**Inner Merge Error, IME**) For $\varphi, \varphi' \in V_S^G$, the inner merge error *IME* (φ, φ') of merging φ and φ' can be defined in Equation 2.

$$IME(\varphi, \varphi') = |FC(\varphi, \varphi')| - |AC(\varphi, \varphi')| \tag{2}$$

Where $FC(\varphi, \varphi') = \{(u, v) \mid \forall u \in \varphi \wedge \forall v \in \varphi'\}$ represents the full connections between two supernode φ and φ'. $AC(\varphi, \varphi') = \{(u, v) \mid \exists u \in \varphi\ \exists v \in \varphi'.(u, v) \in E_G\}$ is the actual connections between the sets.

Definition 5. (**Adjacent Merge Error, AME**) For merging two supernodes $\varphi, \varphi' \in V_S^G$ to form a new supernode φ'', the adjacent merge error *AME* (φ, φ') can be defined in Equation 3.

$$\begin{aligned} AME(\varphi, \varphi') = & \sum_{p \in (N_\varphi \cup N_{\varphi'}) \setminus \varphi''} IME(\varphi'', p) - \\ & \sum_{p \in N_\varphi \setminus \varphi'} IME(\varphi, p) - \sum_{p \in N_\varphi \setminus \varphi} IME(\varphi', p) \end{aligned} \tag{3}$$

In order to make more operational the calculation of AME, we give the theorem 1.

Theorem 1. *The adjacent merge error $AME(\varphi_i, \varphi_j)$ can be formulated by Equation 4.*

$$\begin{aligned} AME(\varphi, \varphi') = & \sum_{p \in (N_\varphi \cup N_{\varphi'}) \setminus (\varphi \cup \varphi')} (IME(\pi(\varphi), p) \\ & - IME(\varphi, p) - IME(\varphi', p)) \end{aligned} \tag{4}$$

Proof. Suppose that φ'' is the resulting supernode after merging φ and φ', i.e., $\varphi'' = \pi(\varphi) = \pi(\varphi')$ and $\varphi'' = \varphi \cup \varphi'$. We have $\sum_{p \in (N_\varphi \cup N_{\varphi'}) \setminus \varphi''} IME(\varphi'', p) = \sum_{p \in (N_\varphi \cup N_{\varphi'}) \setminus (\varphi \cup \varphi')} IME(\pi(\varphi), p)$. Further, because $N_\varphi = \{\varphi' \mid \exists \varphi' \in V_S^G, <\varphi, \varphi'> \in E_S^G \vee <\varphi', \varphi> \in E_S^G\}$, we have $N_\varphi \cap \varphi = \emptyset$ for each possible $\varphi' \in V_S^G$. In the situation, $\sum_{p \in N_\varphi \setminus \varphi'} IME(\varphi, p) + \sum_{p \in N_{\varphi'} \setminus \varphi} IME(\varphi', p) = \sum_{p \in (N_\varphi \cup N_{\varphi'}) \setminus (\varphi \cup \varphi')} (IME(\varphi, p) + IME(\varphi', p))$. According to the above, we know $AME(\varphi, \varphi') = \sum_{p \in (N_\varphi \cup N_{\varphi'}) \setminus \varphi''} IME(\varphi'', p) - \sum_{p \in N_\varphi \setminus \varphi'} IME(\varphi, p) - \sum_{p \in N_{\varphi'} \setminus \varphi} IME(\varphi', p) = \sum_{p \in (N_\varphi \cup N_{\varphi'}) \setminus (\varphi \cup \varphi')} IME(\pi(\varphi), p) - \sum_{p \in (N_\varphi \cup N_{\varphi'}) / (\varphi \cup \varphi')} (IME(\varphi, p) + IME(\varphi', p)) = \sum_{p \in (N_\varphi \cup N_{\varphi'}) / (\varphi \cup \varphi')} (IME(\pi(\varphi), p) - IME(\varphi, p) - IME(\varphi', p))$. □

Definition 6. (**Merge Error, ME**) The merge error $ME(\varphi, \varphi')$ of merging two supernodes φ, $\varphi' \in V_S^G$ to form a new supernode φ'', can be defined as Equation 5.

$$ME(\varphi, \varphi') = AME(\varphi, \varphi') + IME(\varphi, \varphi') \tag{5}$$

The process of merging supernodes is iteratively performed in a map/reduce-like distributed way. Supernodes with the minimum merge error will be preferentially merged. Calculating merge error of two supernodes is parallelly made on Spark in an iterative manner. A merge error threshold ET for each iteration is introduced to handle supernode merge of supernodes. ET is originally set to 0 for first merging supernode pairs with the same

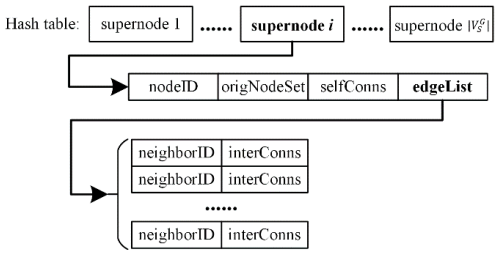

Fig. 5: Hash Table-based Summary Indexing

neighbors. With more and more number of merged supernodes, the merge error will inevitably continue to increase due to the accumulation of the previous merging errors. Thus, the value of ET has to be dynamically adjusted for enabling subsequent supernode merging. ET is adjusted by equation 6.

$$ET_t = ET_{t-1} + 2 * \left\lfloor \frac{|V_G|}{|V_S^G|} \right\rfloor \tag{6}$$

Where $|V_G|$ and $|V_S^G|$ are respectively the numbers of original nodes and the supernodes at the current $(t-1)^{th}$ iteration. During iterations for graph summarization, if ET grows too slow, then finding two appropriate supernodes may be a little difficult. In contrast, if ET grows too fast, then only part of appropriate supernodes may be merged at each iteration. Thus, for efficient node merge, we need to set dynamic error increment threshold at the next iteration based on the previous threshold plus $2 * \left\lfloor \frac{|V_G|}{|V_S^G|} \right\rfloor$, where $\lfloor . \rfloor$ is the integer part of a real number.

4.1.2 Hash Table Structure of RDF Summary Indexing

In this paper, we present a hash table-based summary indexing structure by hashing all the supernodes, whose data structure is shown in Figure 5.

In the hash table-based summary (HTS) indexing structure, each key in the hash table is the unique ID of a supernode. Each supernode also maintains the set of original nodes contained in it, the set of self-connections among them, and the set of adjacent supernodes (neighbors). The specific data structures with HTS are as follows.

node.ID: the unique ID of the specified supernode.

node.origNodeSet: storing all the original nodes contained in the specified supernode.

node.selfConns: maintaining all the original edges among the original nodes contained in the specified supernode. Each actual edge is stored in the form of (s, p, o).

node.edgeList: the set of all the superedges adjacent to the specified supernode.

edge.neighborID: a neighbor supernode ID such that pair $<$ *nodeID*, *neighborID* $>$ can be determined by a specified superedge.

edge.interConns: the set of actual original edges contained in the superedge $<$ *nodeID*, *neighborID* $>$. Similar to selfConn, these actual edges are stored by the form of triple $(s.p, o)$.

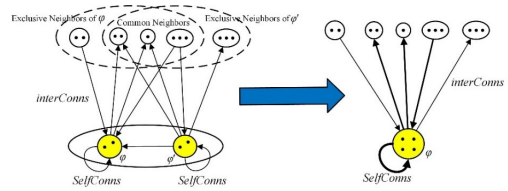

Fig. 6: An illustration of merging supernodes φ and φ'

4.1.3 Graph Summarization-Based Hash Table Indexing

The input of graph summarization is just the original RDF graph that can be also regarded as summary graph. It is stored in HDFS as the set of triples. The original indexing for the original RDF data is built by hashing all the subjects in RDF triples. We also give a predefined k value as input, which is the number of supernodes in the generated summary graph.

Merge of Supernodes. Merge of supernodes is the key step for graph summarization. Figure 6 is an illustrative process when merging supernodes φ and φ'. If merge error of two supernodes φ and φ' is less than the given threshold, both them can be merged and form a new supernode φ''. For convenience, instead of creating a new ID for φ'', we reuse the ID of φ and further merge the sets of original nodes respectively corresponding to φ and φ' into a new set that is taken as the set of original nodes of the newly merged φ''. The connections between φ and φ' are used as the self-connections within φ''.

The process of merging two supernodes is shown in Algorithm 1. We first update the edge information within neighbors of φ and φ'. If a supernode n is the common neighbor of φ and φ', then the actual edges in superedge $<n, \varphi'>$ is added into the set of actual edges corresponding to $<n, \varphi>$. $interConns_{n,\varphi}$ is $HTG(n).edge(\varphi).interConns$ for simplification, and represents the set of actual edges in $<n, \varphi>$. If n is an exclusive neighbor of φ', then no update is made. The next is to update φ and its adjacent superedges. Similarly, $selfConns_{\varphi}$ refers to $HTG(\varphi).selfConns$, representing the set of actual original edges within φ. At last, the hash table HTG is updated by removing the information related to φ'.

Hash Table Indexing Based on Summarization. The whole of lossless graph summarization for building hash table indexing is shown in Algorithm 2, which is made in an iterative way until the number of supernodes in the hash table reaches the predefined value k.

In Algorithm 2, we use $|HTG|$ to represent the total number of supernodes in HTG. We randomly a node ϕ, and obtain its neighbors and two-hop neighbors. Then, we get the supernode φ' that has the largest similarity with φ from neighbors of φ. We use equation 5 to compute the merge error of φ and φ'. If it is less than threshold ET, then Algorithm 1 is invoked to merge φ and φ'. If the number of iterations reaches $|HTG|$, then equation 6 is used for adjusting error threshold. The summary HTG is returned.

Algorithm 1: nodeMerge

input : HTG: hash table of summary;
(φ, φ'):supernode pairs to be merged.
output: HTG: updated hash table.

$N_\varphi \leftarrow \{n | n \in HTG(\varphi).edgeList\}$;
$N_{\varphi'} \leftarrow \{n | n \in HTG(\varphi').edgeList\}$;
for $n \in N_\varphi \cap N_{\varphi'}$ **do**
  $interConns_{n,\varphi} \leftarrow interConns_{n,\varphi} \bigcup interConns_{n,\varphi'}$;
  $HTG(n) \leftarrow HTG(n).edgeList \setminus \{HTG(n).edge(\varphi')\}$;
for $n \in N_{\varphi'} \setminus N_\varphi$ **do**
  $HTG(n).\ edge(\varphi').neighborID \leftarrow \varphi$;
for $n \in N_{\varphi'}$ **do**
  if $n \in N_\varphi \cap N_{\varphi'}$ **then**
    $interConns_{\varphi,n} \leftarrow interConns_{\varphi,n} \cup interConns_{\varphi',n}$;
  else
    $HTG(\varphi).edgeList \leftarrow HTG(\varphi).edgeList \cup HTG(\varphi').edge(n)$;
$selfConns_\varphi \leftarrow selfConns_\varphi \cup selfConns_{\varphi'} \cup interConns_{\varphi,\varphi'} \cup interConns_{\varphi',\varphi}$;
$HTG(\varphi).edgeList \leftarrow HTG(\varphi).edgeList \setminus \{HTG(\varphi).edge(\varphi')\}$;
$\varphi \leftarrow \varphi \cup \varphi'$;
$HTG \leftarrow HTG \setminus \{\varphi'\}$;
return HTG;

Algorithm 2: graphSummarization

input : HTG : hash table of RDF data G; k: the number of supernodes in summary.
output: HTG: hash table of summary graph.

$(t, ET) \leftarrow (0, 0)$;
do
  $\varphi \leftarrow HTG.getRandomNode()$;
  $N_\varphi \leftarrow \{\varphi' | \varphi' \in HTG(\varphi).edgeList\}$;
  $2hopN_\varphi \leftarrow \{\varphi'' \mid \varphi' \in N_\varphi \wedge \varphi'' \in HTG(\varphi').edgeList\} \setminus \{\varphi\}$;
  $sim \leftarrow 0$;
  for $\varphi'' \in 2hopN_\varphi$ **do**
    $N_{\varphi''} \leftarrow \{p | p \in HTG(\varphi'').edgeList\}$;
    if $\frac{|N_\varphi \cap N_{\varphi''}|}{|N_\varphi \cup N_{\varphi''}|} > sim$ **then**
      $sim \leftarrow \frac{|N_\varphi \cap N_{\varphi''}|}{|N_\varphi \cup N_{\varphi''}|}$;
      $\varphi' \leftarrow \varphi''$;
  $ME(\varphi, \varphi') \leftarrow AME(\varphi, \varphi') + IME(\varphi, \varphi')$;
  if $ME(\varphi, \varphi') \leq ET$ **then**
    $HTG \leftarrow nodeMerge(HTG,\ \varphi,\ \varphi')$;
  $t \leftarrow t + 1$;
  if $t = |HTG|$ **then**
    $(t, ET) \leftarrow (0, ET + 2 \times \lfloor \frac{N}{|HTG|} \rfloor)$;
while $|HTG| > k$;
return HTG ;

4.2 Summary-based Vertical Partitioning

We present an RDF summary-based vertical partitioning approach (SumVP) that partitions the summary of the RDF data instead of the original RDF data for significantly diminishing cross-table join operations. In real-world RDF data, the number of predicates is far less than that of subjects and objects, and thus predicates are used to index different binary tables. Specifically speaking, an RDF summary graph $S^G = (V_S^G, E_S^G, P_S^G, \phi_S)$ corresponds to a set of triples $\{(\varphi, p, \varphi') \mid < \varphi, \varphi' > \in E_S^G \wedge p \in \phi_S(< \varphi, \varphi' >)\}$. According to each predicate $p \in P_S^G$, we vertically build its binary table B_p, as shown in Equation 7.

$$B_p = \{(\varphi, \varphi') \mid < \varphi, \varphi' > \in E_S^G \wedge p \in \phi_S(< \varphi, \varphi' >)\} \quad (7)$$

Because S^G is indexed by HTS_G, $\phi_S(<\varphi, \varphi'>)$ can be obtained from HTS_G by Equation 8.

$$\phi_S(< \varphi, \varphi' >) = \begin{cases} \{p \mid (s, p, o) \in \textit{selfConns}_\varphi\}, \\ \quad \textbf{If } \varphi = \varphi'; \\ \{p \mid (s, p, o) \in \textit{interConns}_{\varphi,\varphi'}\}, \\ \quad \textbf{Otherwise.} \end{cases} \quad (8)$$

The set BT of all binary tables is defined by Equation 9.

$$BT = \bigcup_{p \in P_S^G} \{B_p\} \quad (9)$$

The approach to building BTs is described in Algorithm 3. We traverse all the supernodes in the hash table and further add superedges (selfConns and interConns) into the corresponding binary table for each $p \in P_S^G$. We traverse all the superedges for a given suprenode ϕ, and build summary-based binary tables for each predicate. At last, we store the statistics information of table size.

4.3 Binary Tables-Based Relation Pre-joins

A SPARQL query with n pattern triples will have to make relational join operations of $n - 1$ times. For reducing tremendous join operations and eliminating impossible joins, we use a rational join-based BT table preprocessing to build pre-join ternary tables (TT) from BT tables by computing the possible join relations between partitions of BT tables. Each ternary table corresponds to a predicate pair that is used to index the ternary table.

Let $q_i = (s, p_i, o)$ and $q_j = (s', p_j, o') \in QT$ be two query pattern triples. If $s = s'$, then the results of querying q_i and q_j are $B_{p_i} \bowtie_{s=s} B_{p_j}$, i.e., the results of B_{p_i} and B_{p_j} joins with the same subject for predicates p_i and p_j. This kind of joins is called $Subject{-}Subject(SS)$ join, denoted as T_{p_i,p_j}^{SS}, shown in Equation 10.

$$T_{p_i,p_j}^{SS} = \{(s, o, o') \mid (s, o) \in B_{p_i}, (s' \rho') \in B_{p_j}, s = s'\} \quad (10)$$

Algorithm 3: buildBinaryTable

input : HTS_G: hash table of summary S^G.
output: BT: set of binary tables; $btStat$: BT statistics.

```
1  for p ∈ P_S^G do
2      B_p ← ∅;
3  for φ ∈ HTS_G do
4      for p ∈ φ_S( < φ, φ>) do
5          B_p ← B_p ∪ {(φ, φ')};
6      for φ' ∈ HTS_G(φ).edgeList do
7          for p ∈ φ_S( < φ, φ'>) do
8              B_p ← B_p ∪ {(φ, φ')};
9  BT ← ∅ ;
10 for p ∈ P_S^G do
11     BT ← BT ∪ {B_p} ;
12     btStat ← btStat ∪ {|B_p|};
13 return (BT, btStat);
```

Besides SS joins, there may have other kinds of joins including $Subject-Object(SO), Object-Subject(OS)$, and $Object-Object(OO)$. Their definitions refer to equations 11-13.

$$T^{SO}_{p_i,p_j}=\{(s,s',o')|\ (s,o)\in B_{p_i},(s',o')\in B_{p_j},s=o'\} \quad (11)$$

$$T^{OS}_{p_i,p_j}=\{(s,o,o')|\ (s,o)\in B_{p_i},(s',o')\in B_{p_j},o=s'\} \quad (12)$$

$$T^{OO}_{p_i,p_j}=\{(s,o,s')|\ (s,o)\in B_{p_i},(s',o')\in B_{p_j},o=o'\} \quad (13)$$

Theorem 2. *Half of cross table joins can be calculated for all the pre-joins between binary tables.*

Proof. Without loss of generality, let B_{p_i} and B_{p_j} be the binary tables respectively corresponding to predicates p_i and p_j. First, the SS joins between B_{p_j} and B_{p_i} can be represented by $T^{SS}_{p_j,p_i}=\{(s',o',o)\mid(s',o')\in B_{p_j},(s,o)\in B_{p_i},s'=s\}=\{(s,o',o)\mid(s,o)\in B_{p_i},(s',o')\in B_{p_j},s=s'\}=T^{SS}_{p_i,p_j}$. Therefore, we know that the SS join for predicate pair $(p_j,\ p_i)$ can be obtained by the SS join for predicate pair (p_i,p_j). The SS joins for pairs (p_i,p_j) and (p_j,p_i) have the same results.

Second, similarly for the SS joins, we have $T^{OO}_{p_j,p_i}=\{(s',o',s)\mid(s',o')\in B_{p_j},(s,o)\in B_{p_i},o'=o\}=\{(s',o,s)\mid(s',o')\in B_{p_j},(s,o)\in B_{p_i},o=o'\}=T^{OO}_{p_i,p_j}$.The results of OO joins for pairs (p_i,p_j) and (p_j,p_i) are the same.

Third, the SO join for predicates p_j and p_i, $T^{SO}_{p_j,p_i}=\{(s',o',s)\mid(s',o')\in B_{p_j},(s,o)\in B_{p_i},s'=o\}=\{(o,o',s)\mid(s,o)\in B_{p_i},(s',o')\in B_{p_j},o=s'\}=T^{OS}_{p_i,p_j}$. We can find that the results of the SO and OS joins respectively for predicate pairs (p_i,p_j) and (p_j,p_i) are the same, vice versa.

From the above, we can find that we only need to calculate the half of cross table joins for building all the four kinds of pre-joins between binary tables. □

According to the theorem 2, we only calculate the half of SS joins and OO joins, plus all the OS joins for obtaining the TT, which is defined in equation 14.

$$TT=\bigcup_{p_i,p_j\in P_G\wedge i\le j}\left\{T^{SS}_{p_i,p_j},T^{OO}_{p_i,p_j}\right\}\cup\bigcup_{p_i,p_j\in P_G}\left\{T^{OS}_{p_i,p_j}\right\} \quad (14)$$

Building TT tables for pre-join optimization of SPARQL queries is shown in Algorithm 4. We calculate the joins with respect to the pre-join types (i.e., SS, OO and OS), and avoid repeatedly calculation (lines 3-12). At lines 13-14, the pre-joins are stored in ternary tables. We preserve possible ternary tables for statistics-based query optimization. There may exist empty ternary table when no any join type is found between two BTs. If a SPARQL query evaluation deals with empty ternary tables in the future, then query results can be directly returned empty instead of performing unnecessary joins.

Algorithm 4: buildTernaryTable

input : BT: set of binary tables.
output: TT: set of ternary tables; $ttStat$: TT statistics.

```
1  for p_i ∈ P_S^G do
2      for p_j ∈ P_S^G do
3          for PreJ ∈ {SS, OS, OO} do
4              switch PreJ do
5                  case SS do
6                      if i ≤ j then
7                          tab ← {(s, o, o') | (s, o) ∈
                             B_pi, (s', o') ∈ B_pj, s = s'};
8                  case OO do
9                      if i ≤ j then
10                         tab ← {(s, o, s') | (s, o) ∈
                             B_pi, (s', o') ∈ B_pj, o = o'};
11                 case OS do
12                     tab ← {(s, o, s') | (s, o) ∈
                             B_pi, (s', o') ∈ B_pj, o = o'};
13             ttStat ← ttStat ∪ {|tab|};
14             if tab ≠ ∅ then TT ← TT ∪ {tab};
15 return (TT, ttStat );
```

5 SPARQL Query Optimization and Evaluation

5.1 Statistics-Based Optimal Order of Query Triples

Although changing the order of cross-table joins ultimately does not change the query results, different orders will generate different size of intermediate results [42]. We have to use an optimal order of cross-table joins for producing as less intermediate results as possible. For any

two tables tab and tab', the size of their join results is $|tab \bowtie tab'| \leq |tab| \times |tab'|$. The smaller table size is, the less the intermediate results are. So we first need to join tables that have smaller sizes for diminishing intermediate results.

Definition 7. (**Bound Number of Pattern Triple, BN**) The bound number BN_q of a query pattern triple $q = (s, p, o) \in QT$ is the number of bound elements in q, which is defined in Equation 15.

$$BN_q = 2 - |Vars(\{q\})| \tag{15}$$

Where $Vars(TS)$ is the set of all the variables in query triple set TS, and $|Vars(\{q\})| \leq 2$. It is not difficult to find that a query pattern triple with smaller larger bound number will have less query results for the same binary table.

So we present a strategy for optimizing the order of pattern triples during SPARQL query evaluation: 1) the query pattern triples with smaller bound numbers should be preferentially performed; and 2) if two pattern triples have the same BN value, the triple whose predicate pertains to the smaller size of BT will be preferentially performed. Formally speaking, For two pattern triples $q, q' \in QT$, the preference of performing q is higher than that of q' if and only if the following conditions hold:

1) $BN_q > BN_{q'}$; or

2)$|B_p| < |B_{p'}|$ if $BN_q = BN_{q'}$.

We use $W^{BT}(q)$ to represent the preference of $q \in QT$ over BT, which is defined by equation 16.

$$W^{BT}(q) = BN_q + \frac{1}{|B_p|} \tag{16}$$

Correspondingly, the preference of performing triples q and q' w.r.t the join type $PreJ \in \{SS, OS, OO\}$ over TT is defined in equation 17.

$$W^{TT}(q, q', PreJ) = BN_q + BN_{q'} + \frac{1}{|T^{PreJ}_{p,p'}|} \tag{17}$$

Because a ternary table is built by pre-joins over BTs with respect to some join type $PreJ \in \{SS, OS, OO\}$, there possibly exists a empty ternary table $T^{PreJ}_{p,p'} \in TT$ such that $|T^{PreJ}_{p,p'}| = 0$. To make sure that equation (17) is valid in this situation, we set $W^{TT}(q, q', PreJ) = 6$ due to the maximum $W^{TT}(q, q', PreJ) = 5$ if $|T^{PreJ}_{p,p'}| \neq 0$.

For any pattern triple $q = (s, p, o) \in QT$, its neighbor triples $NT_q = \{(s', p', o') \in QT \setminus \{q\} \mid \{s, o\} \cap \{s', o'\} \neq \emptyset\}$. Based on TT, we pre-compute the possible joins between neighbor triples for diminishing unnecessary joins. Our SPARQL query optimization is shown in Algorithm 5. We first sort triples in QT in terms of their preferences (lines 2-7). The triple with the maximum preference is put into $orderedQT$ for each iteration. Then, if $|QT|$=1, we return results without performing remaining statements. If the SPARQL query includes multiple triples, we select a triple q' with the maximum weight from its neighbor triples NT_q as its partner for the currently unselected triple q in $orderedQT$ (lines 10-21). The pre-join type and order between q and q' are also preserved in $queryPreJ$.

Algorithm 5: optimizeOrderOfQueriyTriples

input : QT: set of query triples.
output: $orderedQT$: optimal order of query triples; $queryPreJ$: set of pre-joins.

1 $orderedQT \leftarrow \emptyset$;
2 **while** $QT \neq \emptyset$ **do**
3 $\quad maxW \leftarrow 0$;
4 $\quad$ **for** $q \in QT$ **do**
5 $\quad\quad$ **If** $maxW < W^{BT}(q)$ **then** $(maxW, t) \leftarrow (W^{BT}(q), q)$;
6 $\quad orderedQT \leftarrow orderedQT \cup \{t\}$;
7 $\quad QT \leftarrow QT \setminus \{t\}$;
8 **if**$|QT| = 1$ **then return** ($orderedQT$,$\emptyset$);
9 $(queryPreJ, selectedQT) \leftarrow (\emptyset, \emptyset)$;
10 **for** $q = (s, p, o) \in orderedQT$ **do**
11 $\quad$ **if** $selectedQT \cap \{q\} = \emptyset$ *and* $NT_q \setminus selectedQT \neq \emptyset$ **then**
12 $\quad\quad NT_q \leftarrow NT_q \setminus selectedQT$;
13 $\quad\quad maxW \leftarrow 0$;
14 $\quad\quad$ **for** $q' = (s', p', o')$ *in* NT_q **do**
15 $\quad\quad\quad$ **if** $s = s' \wedge maxW < W^{TT}(q, q', SS)$ **then** $(maxW, PreJ) \leftarrow (W^{TT}(q, nt, SS), SS)$;
16 $\quad\quad\quad$ **If** $o = s' \wedge maxW < W^{TT}(q, q', OS)$ **then** $(maxW, PreJ) \leftarrow (W^{TT}(q, q', OS), OS)$;
17 $\quad\quad\quad$ **if** $s = o' \wedge maxW < W^{TT}(q', q, OS)$ **then** $(maxW, PreJ) \leftarrow (W^{TT}(q', q, OS), SO)$;
18 $\quad\quad\quad$ **if** $o = o' \wedge maxW < W^{TT}(q, q', OO)$ **then** $(maxW, PreJ) \leftarrow (W^{TT}(q, q', OO), OO)$;
19 $\quad\quad$ **if** $PreJ = SO$ **then** $queryPreJ \leftarrow queryPreJ \cup \{(q', q, OS)\}$;
20 $\quad\quad$ **else** $queryPreJ \leftarrow queryPreJ \cup \{(q, q', PreJ)\}$;
21 $\quad\quad selectedQT \leftarrow selectedQT \cup \{q, q'\}$;
22 **return** $(orderedQT, queryPreJ)$;

Here, we illustrate an example of query optimization in Figure 7. We use the SPARQL query in Figure 3, and the query is performed over the original RDF data in Figure 1. In Figure 7(b), the statistics values related to SV, BN, BT and TT for each triple in QT are first obtained. Equation 16 is used to compute preference value W. Here, SV is the number of tuples in a table tab, i.e., $SV(tab) = |tab|$. Figure 7(c) is used to sort the triples by their preference weights. In Figure 7(d), the SPARQL query of sorted triples is translated into Spark SQL language. Figure 7(e) describes the TT statistics-based statistics information. The neighbor triples of pattern triple q_1 are q_2 and q_3 with join type

SS ($PreJ = SS$). We calculate $W^{TT}(q_1, q_2, SS)$=2 and $W^{TT}(q_1, q_3, SS)$=2.33 by Equation 17. Then, in figure 7(f), $TT^{SS}_{1,2}$ is the ternary table with SS join for predicates in triples q_1 and q_2. We select the triple join preference q_1 and q_3 with SS join type because $W^{TT}(q_1, q_3, SS)$ has the large value than $W^{TT}(q_1, q_2, SS)$, where BN is calculated by $BN_{q_1} + BN_{q_2}$ or $BN_{q_1} + BN_{q_3}$ and SV is calculated by $|TT^{SS}_{1,2}|$ or $|TT^{SS}_{1,3}|$. The Spark SQL statement over summary is generated (Figure 7(g)), where π(GraduateStudent) refers to the supernode in the summary pertaining to the original node GraduateStudent, and advisor_workFor_OS is the ternary table with OS join type corresponds to predicates advisor and workFor.

5.2 Summary Recovery-Based Query Evaluation

In this paper, we translate SPARQL queries into the equivalent Spark SQL statement for query evaluation over Spark. All the variables in the ordered query triples are placed behind SELECT, BN values behind WHERE, and all the table names behind FROM. The translation from SPARQL queries to Spark SQL ones is shown in Algorithm 6, where all variables are put into projections, and the bound elements are added into conditions. Here, we use relational algebra notation to represent the Spark SQL query statement $sparkSQL$, which will be performed over Table tab by Spark's built-in function $SPARK.querySQL(sparkSQL)$ for obtaining the query results. When $forSum$ is true, $sparkSQL$ will be performed over a summary table (i.e., BT or TT). Otherwise, it will be performed over the intermediate results produced by summary recovery.

Algorithm 6: SPARQL2SparkSQL($qtSet$, $forSum, tab$)

input : $qtSet$: query triples set; $forSum$: boolean value; tab : query table.
output: $sparkSQL$: Spark SQL statement.

$(projections,\ conditions) \leftarrow (\emptyset, \emptyset)$;
for $q = (s, p, o) \in qtSet$ **do**
  $projs \leftarrow projs \cup Vars(q)$;
  $conds \leftarrow conds \cup (\{s, o\} \setminus Vars(q))$;
if $forSum$ **then**
  $conds \leftarrow \{\pi(c) | c \in conds\}$;
return $sparkSQL \leftarrow \Pi_{projs}(\sigma_{conds}(tab))$;

What is worth noting that the query results of performing **SPARQL2SparkSQL**($qtSet$, $forSum, tab$) can be also regarded as a temporary table, where the number of columns is $|Vars(qtSet)|$ and every row is a query result satisfying the query condition.

We first perform SPARQL queries over the vertically partitioned tables of the RDF summary. Then the query results based on the summary are reversely tracked for obtaining the intermediate results from the original RDF data, where every supernode is corresponded to a set of original nodes. However, the intermediate results produced by summary-based query evaluation may include tremendous spurious results, so we conduct summary-based result recovery to filter the spurious intermediate results by using the Hash table-based summary indexing HTG mentioned in Algorithm 2. We use $conns_{\varphi,\varphi'}(p)$ to represent the set of actual edges contained in superedge $< \varphi, \varphi' >$ w.r.t predicate p. It is defined in Equation 18.

$$conns_{\varphi,\varphi'}(p) = \begin{cases} \{(s, o) \mid (s, p, o) \in \textit{interConns}_{\varphi,\varphi'}\}, \\ \quad \textbf{if } \varphi \neq \varphi' \\ \{(s, o) \mid (s, p, o) \in \textit{selfConns}_{\varphi}\}, \\ \quad \textbf{otherwise} \end{cases} \tag{18}$$

Where $interConns_{\varphi,\varphi'}$ and $selfConns_{\varphi}$ are actual edges respectively from superedges $<\varphi, \varphi'>$ and $<\varphi, \varphi>$.

We use $Recov(q, SR)$ to represent the correct intermediate results w.r.t the pattern triple $q = (s, p, o)$ by using summary recovery to filter the spurious intermediate results from the query results SR based on the summary, which is formally defined in Equation 19.

$$Recov((q, SR)) = \begin{cases} \{conns_{\varphi,\varphi'}(p) \mid (\varphi, \varphi') \in \Pi_{s,o}(SR)\}, \\ \quad \textbf{if } s, o \in Vars(\{q\}); \\ \{conns_{\varphi,\pi(o)}(p) \mid \varphi \in \Pi_s(SR)\}, \\ \quad \textbf{if } s \in Vars(\{q\}) \wedge o \notin Vars(\{q\}); \\ \{conns_{\pi(s),\varphi}(p) \mid \varphi \in \Pi_o(SR)\}, \\ \quad \textbf{if } s \notin Vars(\{q\}) \wedge o \in Vars(\{q\}); \\ \{\text{conns}_{\pi(s),\pi(o)}(p)\}, \textbf{otherwise.} \end{cases} \tag{19}$$

Where $\Pi_c(SR)$ is a projection operation with property column set c over the table SR.

For a simple SPARQL query QT with a single pattern triple (i.e., $|QT| = 1$), we only perform QT over binary tables without ternary tables due to that there is no need to make pre-joins. However, when we perform complex SPARQL queries with multiple pattern triples, we further need to access the pre-joins over tenary tables, which can diminish some impossible joins for accelerating queries. Algorithm 7 describes the whole process of perform complex SPARQL queries. We first get the Spark SQL statements by the pre-joins over ternary tables (lines 1-4). At line 2, if $T^{PreJ}_{p,p'} \notin TT$, then the pattern triples in $orderedQT$ are impossible to have any $PreJ$ pre-join and their query conditions cannot be satisfied, so the program terminates with an empty query result set. Otherwise, the generated Spark SQL statement is submitted for Spark query execution, and the summary-based query results $sumRes$ are obtained (line 5). The summary-based intermediate results recovery is made for filtering spurious joins in the intermediate results (line 8). At lines 7-11, we get the Spark SQL statement over the recovered data and execute it.

An example for performing Spark SQL statement is illustrated in Figure 8, where "(x)-{y}" is used for describing the association between supernode x and original node set $\{y\}$. We suppose that all the relevant tables (e.g., TT and BT) and HTS have been built. First, we obtain Spark SQL query results based on the built binary tables and ternary tables form the summary. Then, we use

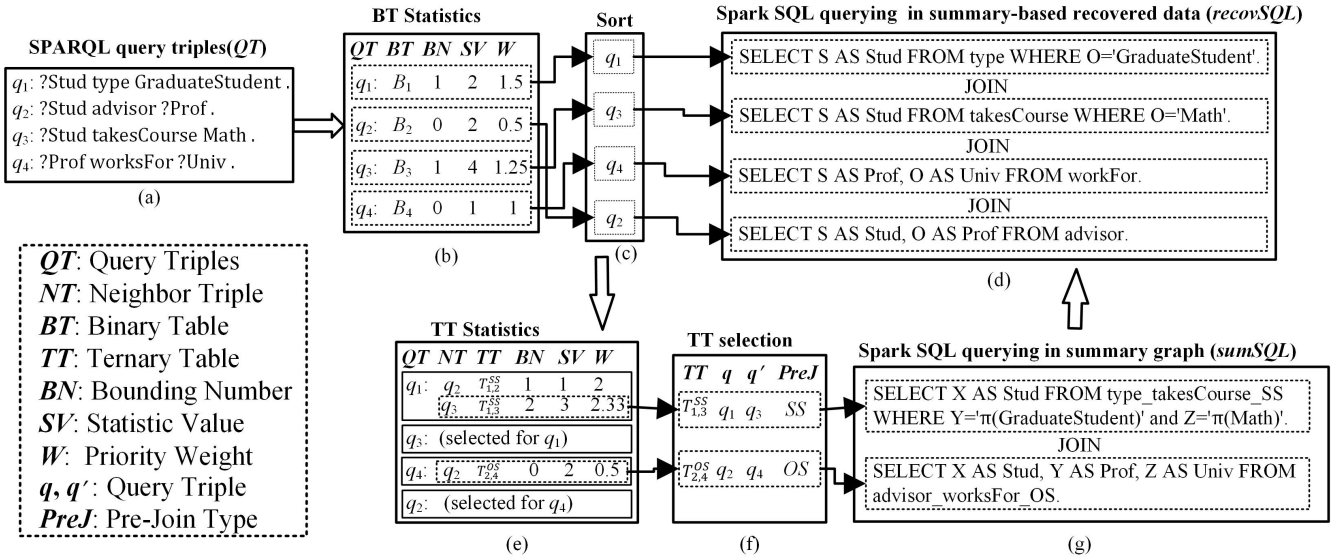

Fig. 7: SPARQL query optimization and Spark SQL translation

Algorithm 7: queryEvaluation

input : $orderdQT$: optimal order of query triples with $|orderdQT| > 1$; $queryPreJ$: set of pre-joins; TT: set of ternary tables.
output: $qRes$: query results.

for $(q, q', PreJ) \in queryPreJ$ **do**
  if $T^{PreJ}_{p,p'} \notin TT$ **return** $\emptyset$;
  if $sumSQL = \emptyset$ **then** $sumSQL \leftarrow SPARQL2sparkSQL(\{q, q'\}, true, T^{PreJ}_{p,p'})$;
  else $sumSQL \leftarrow sumSQL \bowtie SPARQL2sparkSQL(\{q, q'\}, true, T^{PreJ}_{p,p'})$;
$sumRes \leftarrow SPARK.querySQL(sumSQL)$;
$recovSQL \leftarrow \emptyset$;
for $q \in orderedQT$ **do**
  $sumRecov \leftarrow Recov(q, sumRes)$;
  if $recovSQL = \emptyset$ **then** $recovSQL \leftarrow SPARQL2sparkSQL(\{q\}, false, sumRecov)$;
  else $recovSQL \leftarrow recovSQL \bowtie SPARQL2sparkSQL(\{q\}, false, sumRecov)$;
$queryRes \leftarrow SPARK.querySQL(recovSQL)$;
return $queryRes$;

summary hash indexing structure to find the associations between each supernode in the results and their original nodes. We further recover the actual joins by finding the actual edges preserved in the indexing, where spurious joins introduced by calculating full connections for graph summarization had been excluded in the indexing. At last, over the recovered data, we perform the Spark SQL queries translated from the ordered pattern triples for obtaining the final results.

6 Experiments Evaluation and Analysis

6.1 Experiments Settings

We have implemented the SumVP approach proposed over the Spark platform in this paper for accelerating complex SPARQL queries on the Spark/HDFS platform, which is publicly available from the Github website[6]. The Spark cluster was deployed on six worker nodes. Hardware: CPU Intel-i5-10400/6 cores/2.90GHz, Memory 64GB, Disk Drive 4TB. Software: OS CentOS 7.9, JDK 1.8.0_221, Scala 2.11.12, Hadoop 2.6.7, and Spark 2.4.4. We repeated each of experiments 3 times and collected their average values as results.

6.2 Description of Datasets

In this paper, four large-scale datasets with different size respectively from Lehigh University Benchmark (LUBM) and Waterloo SPARQL Diversity Test Suite (WatDiv) were used for experiments and evaluation. LUBM [43] is the RDF data about universities, students, courses, professors and schools. It contains different sizes of RDF data for experiments and evaluation and provides 14 basic query use cases including four types of basic graph pattern (BGP) as well as two single patterns (i.e., graph pattern with one triple). In this paper we use the specific datasets LUBM1k and LUBM5k. WatDiv [44] provides plenty of query use cases through which different queries can be evaluated in a fine-grained manner. WatDiv includes 20 predefined query use cases (basic testing use cases) that can be grouped into four categories according to their

6. https://github.com/LLLll1-dot/SumVP

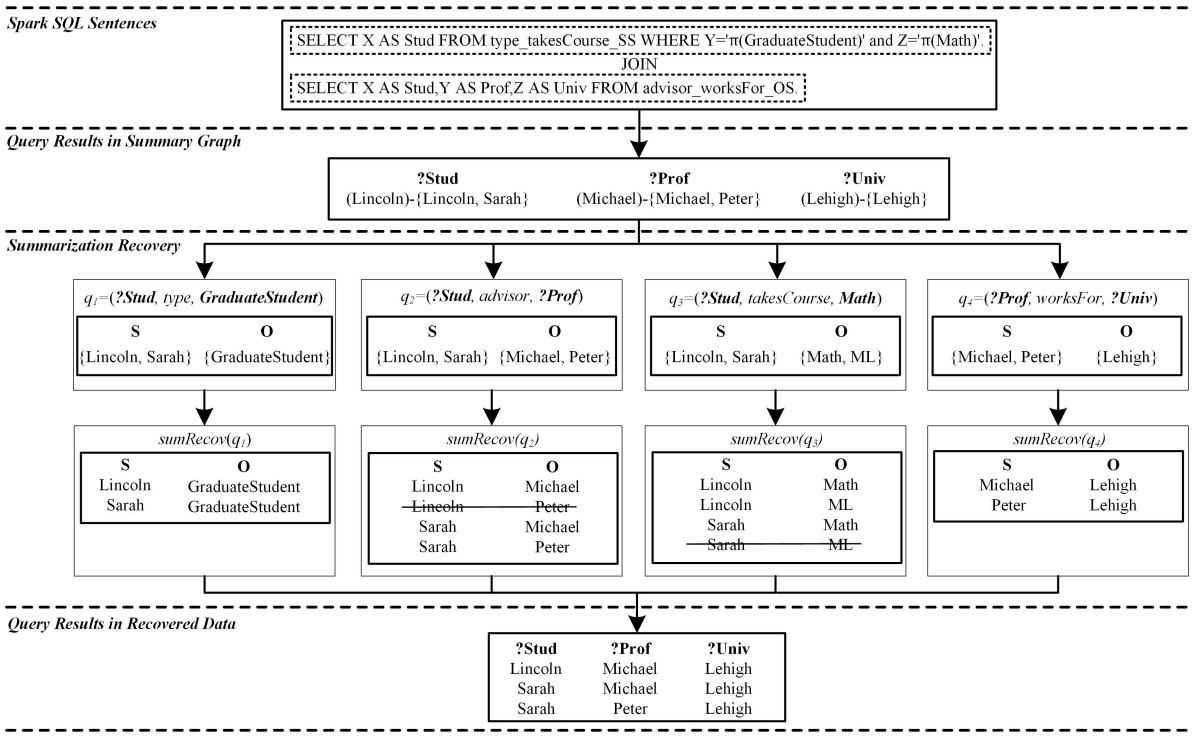

Fig. 8: An example for performing Spark SQL query

TABLE 1: Description of RDF datasets

Dataset	Number of Triples	Number of Predicates
LUBM 1k	138,280,374	17
LUBM 5k	690,895,862	17
WatDiv 100M	108,997,714	86
WatDiv 500M	549,597,531	86

shapes: Star (S), Linear (L), Snowflake (F) and Complex (C). Here, WatDiv100M and WatDiv500M are used for experiments and evaluation. The description of datasets is shown in Table 1.

6.3 Baseline Approaches

To date, there have been dozens of RDF stores and SPARQL engines for large RDF data [42, 45]. According to the categories of query engines in [42], our approach can fall into some technique categories such as *vertical partition*-based storage, hash table-based summary *structural indexing*, *pairwise/multiway joins* and *join reordering*, and spark SQL-based *relational algebra level query rewriting* over Spark and HDFS. Therefore, the state-of-the-art approaches most relevant to our approach are S2RDF [24], S3QLRDF [21, 30], SPARQLGX [26] and CliqueSquare [40]. We selected them as baselines for comparing and evaluating our approach for RDF data storage, processing and queries over Spark and HDFS.

6.4 Experiments and Analysis

Our summary vertical partitioning and query join optimization-based approach will be evaluated over four benchmark datasets in comparison with the state-of-the-art approaches. In the followings, our experiments and analysis were made from four perspectives. First, we will compare the loading time and HDFS store size of datasets for different approaches. Second, we made evaluation for all basic RDF query patterns including Star (S), Linear (L), Snowflake (F) and Complex (C), as well as the pattern with a single triple. The third is about the incremental linear testing. The last is to validate the influence of different summary size on query average mean runtime.

6.4.1 Loading Time and HDFS Store size

The loading times and HDFS store sizes of our SumVP approach as well as the others is listed in Table 2. Because SumVP and S2RDF need to store all the possible pre-joins over the partitioned binary tables, they require more storage space and also take more time to load data. Although SumVP uses full relational joins instead of semi-joins for S2RDF, the required store size of SumVP is slightly less than S2RDF because SumVP stores the pre-joins of RDF summary graph rather than that of the whole raw RDF data. The loading time of SumVP is much less than that of S2RDF. CliqueSquare needs the maximum storage space due to that it duplicates three copies of the raw RDF data. S3QLRDF is based on property tables and has the mediate loading time and store sizes. Due to that SPARQLGX is only based on simple vertical partitioning without pre-

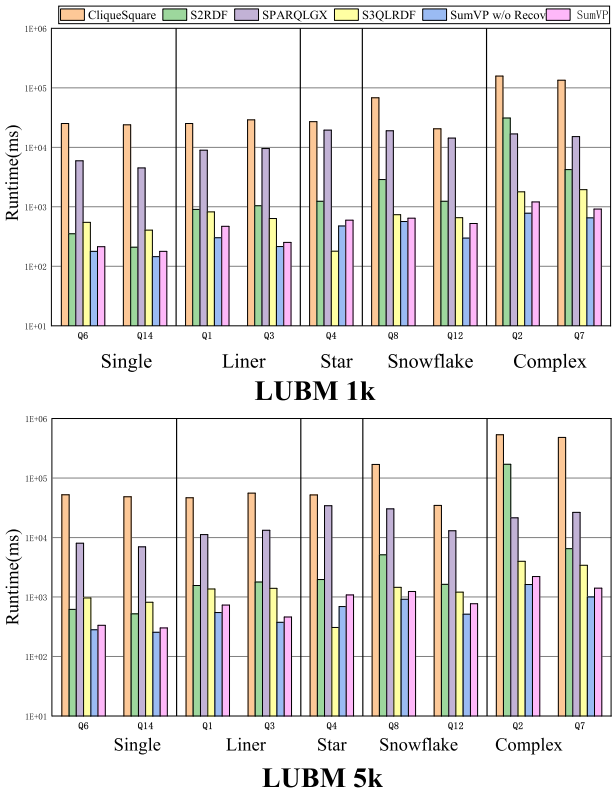

Fig. 9: Average runtimes of evaluating BGP queries over LUBM datasets (ms)

joins and therefore it has the minimum loading time and store size.

6.4.2 Basic Graph Patterns (BGP)

In this section, we conducted RDF queries for basic graph patterns over two groups including the four datasets with different sizes: LUBM1k, LUBM5k and WatDiv100M, WatDiv500M. The experimental results are shown in Figures 9 and 10, where SumVP is our approach, and the SumVP w/o Recov refers to the situation where we evaluate our SPARQL queries without using summary recovery.

In Figure 9, we observe that our approach SumVP outperforms the approaches S2RDF, SPARQLGX and CliqueSquare for all shapes of queries. For example, for query Q2, our approach is approximately 24, 22, 10 and 2 times faster than CliqueSquare, S2RDF, SPARQLGX and S3QLRDF, respectively. Most importantly, with the increasing number of triples in datasets, the performance difference between ours and these approaches becomes more significant. We also find that the three methods SumVP, S2RDF and S3QLRDF translate SPARQL to Spark SQL for evaluating queries, and they have consumed much lower runtimes than that of CliqueSquare and SPARQLGX because both of the latter need to take more time for dynamically generating the distributed query plans and execution commands over partitions of the raw graph data. Furthermore, both S2RDF and S3QLRDF use vertical partitioning based on the raw RDF data instead of their RDF summaries. They produce more intermediate results and need more runtime to perform cross-table join operations than SumVP. S3QLRDF shows an excellent performance to star pattern queries because it is based on property tables. Nevertheless, our approach is also superior to S3QLRDF with exception to the star pattern query Q4 because S3QLRDF takes much irrelevant data as the input (e.g., columns irrelevant to queries in property tables), which increase the overhead of queries. Especially for complex shapes of queries such as Snowklake and Complex, our approach is superior to the other approaches. From the figure 9, we also observe that the summary recovery-based query evaluation (SumVP) has consumed more time than query evaluation without summary recovery because it needs to take extra time to filter spurious joins from the intermediate re-sults, but we argue that summary recovery is necessary for obtaining correct query results to recover the actual joins from the intermediate results.

In Figure 10, we list the average mean runtimes over WatDiv for all the BGP testing use cases. Our approach SumVP overall is superior to all the baseline approaches with exception to the star shape queries for S3QLRDF. With the increasing number of triples in datasets, the performance difference between ours and these approaches becomes more significant. SumVP, S2RDF and S3QLRDF have consumed much lower runtimes than that of CliqueSquare and SPARQLGX. S3QLRDF shows an excellent performance to star pattern queries. From figures 9 and 10, we can find that our approach does not favor any specific shape of query patterns and can achieve consistent query performance over these datasets for BGP queries.

What is worth noting that we also notice that our approach SumVP outperforms these approaches when evaluating complex graph pattern over these datasets. Especially for complex shapes of queries such as Snowklake and Complex, our approach is superior to the other approaches over the two datasets. In contrast to BGP, complex graph pattern [46] allows the tabular results of one or more BGP patterns to be transformed using the relational algebra under the support of SQL-like query languages by operators such as projection, selection, WHERE, union and joins, and so on. In our approach, SPARQL queries are rewritten into Spark SQL queries, which supports relational algebra level tabular queries. We argue that our approach uses relational algebra-based tabular querying on top of the Spark SQL language, which can be naturally used to efficiently handle for complex graph pattern queries.

6.4.3 Incremental Linear Testing

Incremental Linear (IL) testing can be used to validate whether complex SPARQL queries with larger diameter perform well across different nodes in the same cluster. Complex SPARQL queries with complex structures are essentially compositions of some fundamental patterns such as star, linear and snowflake, which often have bigger

TABLE 2: Datasets loading times and HDFS sizes for SumVP and competitors

	Dataset	LUBM 1k	LUBM 5k	WatDiv 100M	WatDiv 500M
HDFS Size	Original	24.0 GB	116.0 GB	15.0 GB	76.0 GB
	CliqueSquare	39.8 GB	203.0 GB	30.3 GB	155.0 GB
	S2RDF	4.9 GB	24.8 GB	11.1 GB	61.8 GB
	SPARQLGX	1.2 GB	12.2 GB	0.9 GB	4.8 GB
	S3QLRDF	4.1 GB	20.9 GB	7.1 GB	36 GB
	SumVP	4.4 GB	21.4 GB	12.7 GB	64.2 GB
Loading Time	CliqueSquare	2876 s	15242 s	3182 s	17015 s
	S2RDF	7132 s	13314 s	9474 s	25388 s
	SPARQLGX	1843 s	4651 s	1692 s	4040 s
	S3QLRDF	1473 s	4044 s	4531 s	8007 s
	SumVP	1663 s	4596 s	3544 s	10121 s

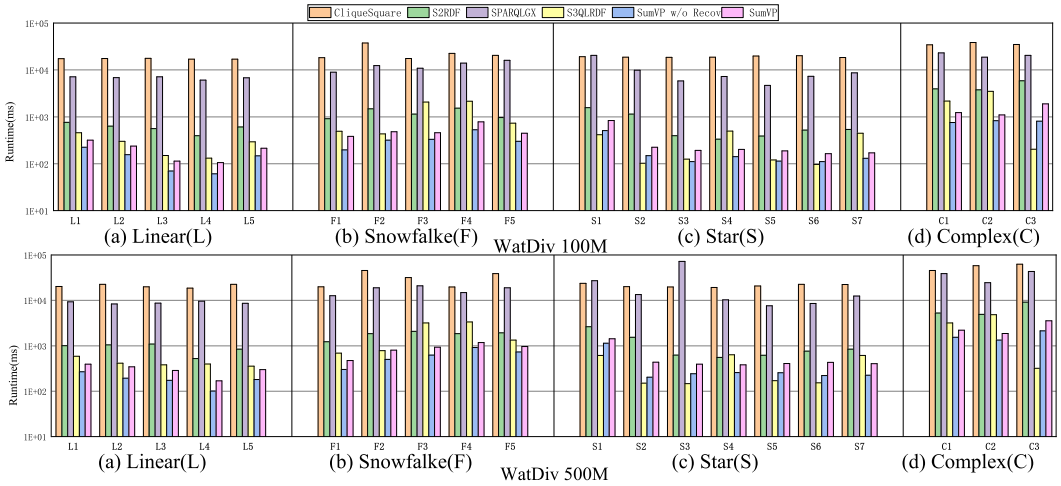

Fig. 10: Average runtimes of evaluating BGP queries over WatDiv datasets (ms)

diameters. Here, the diameter is defined as the longest path in the query pattern graph, i.e., $\max(\{sp(v_i, v_j) \mid v_i, v_j \in V_Q\})$, where $sp(v_i, v_j)$ represents the shortest path between nodes v_i and v_j in the query pattern graph. In BGP, a star-shaped query has diameter 1, and the diameter of a linear shape query is number of edges within it.

We conducted IL testing [44] for validating the performance of SPARQL queries with increasing diameter over the datasets WatDiv100M and WatDiv500M due to that WatDiv datasets provide some IL testing use cases. In WatDiv, an IL testing use case is represented by the form of "IL-x-y", where IL-x represents different shapes of queries and y is the diameter of the query. The IL testing use cases for the WatDiv dataset includes three categories including User Bound (IL-1), Retail Bound (IL-2) and Unbound (IL-3), each of which continually increases their diameters from 5 to 10. The experimental results for IL testing are shown in Figure 11.

We observe that our approach outperforms all the other approaches in terms of all types of IL testing. For a given query pattern, our approach can utilize its possible pre-joins in ternary tables for querying evaluation without scanning all possibly irrelevant joins, and therefore it can significantly reduce querying times. In contrast, although S2RDF also uses the pre-joins between binary tables and has shown the lower runtimes among these approaches, the pre-joins operations of S2RDF is made based on the original RDF data to perform much more joins, which will consume more time. In addition, we observe that the average runtimes of our approach for IL-3 queries are much more than that for IL-1 and IL-2 queries. Because IL-3 queries are unbounded, the intermediate results as well as the ultimate results will much larger than that of the previous two kinds of queries, which means that our approach has to take more time for recovering them and obtaining querying results. Importantly, we also find that our approach performs better over the large-size WatDiv500M than the small-size WatDiv100M due to that it has relatively low ratio of average mean runtimes to data sizes when testing is made over the large dataset. The IL testing over our approach has illustrated superior scalability to complex SPARQL queries with larger diameters.

6.4.4 Influence of Different Graphs on Summary Size

In this section, we will validate the influence of different graphs on summary size for determining a reasonable compression rate. We introduce compression rate that is defined by $|V_S^G|/|V_G|$ as an indicator of summary size, and validate the influence of different graphs on summary sizes in terms of query runtimes. The experiments were made over two datasets LUBM5k and WatDiv500M, shown in Figure 12. The x-axis is compression rate, and

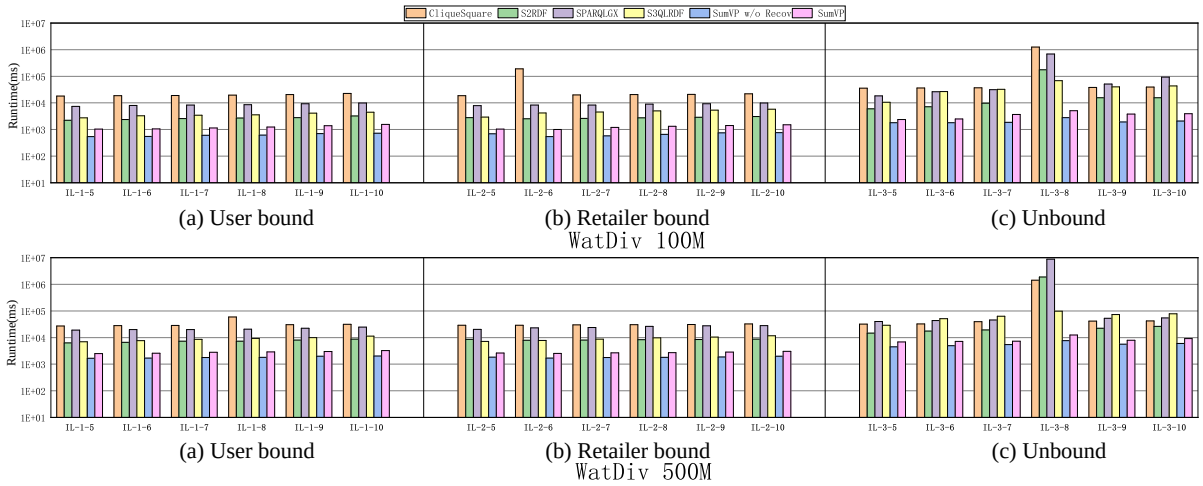

Fig. 11: Incremental Linear Testing

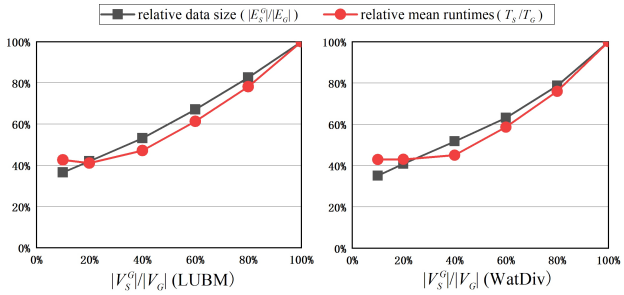

Fig. 12: Influence of different graphs on summary size

the y-axis are respectively the rates of relative edge data size ($|E_S^G|/|E_G|$) and relative mean runtime (T_S/T_G), where T_S and T_G are respectively the average mean runtimes of query evaluation over the current summary and the original RDF graph. From Figure 12, we observe that both the relative edge data size and the relative mean runtimes will dramatically increase with the increasing compression rate. When the compression rate approximately reaches about 10%-20%, our approach can achieve superior performance for evaluating and accelerating complex SPARQL queries, where both the relative edge data size and the relative mean runtimes have the lowest values.

7 Conclusion

In this paper, we present a holistic framework based on summary-based vertical partitioning and query optimization for accelerating complex SPARQL queries over large scale of RDF data on Spark. It does not favor any specific shape of query patterns and achieves consistent query performance. We have conducted extensive experiments over four different datasets in comparison with the state-of-the-art approaches S2RDF, S3QLRDF, SPARQLGX and CliqueSquare, using different query shapes. The results show that our approach is very competitive in comparison with the state-of-the-art distributed SPARQL query approaches on Hadoop/Spark for accelerating complex SPARQL queries, especially for complex shapes of SPARQL queries. It also shows that the compression rate within 10% to 20% for graph summarization achieves superior performance.

We would like to discuss the limitations of our approach and the future work. First, our approach is currently based on static graphs where nodes and edges are not dynamically updated over time. When our lossless graph summarization proposed in this paper is applied to dynamic graph, it has to traverse all the nodes and edges of the whole big graph. But in the real world, the updated information in the graph is incrementally made. For each time of updates, the updated information is often a very small part of the overall graph data. Currently, many works have been also developed for handling summarization and clustering over dynamic graph data [47–51]. Considering that our summarization approach is offline made and loosely coupled to other modules in the framework, we in the future will concentrate on handling dynamic updates and dynamic indexing structure. Second, the benchmark datasets used in our experiments are synthetic graph data. We will evaluate our approach over some real-world datasets such as DBpedia for validating the performance of our approach.

References

[1] D. Song, F. Schilder, S. Hertz, G. Saltini, C. Smiley, P. Nivarthi, O. Hazai, D. Landau, M. Zaharkin, T. Zielund, H. Molina-Salgado, C. Brew, and D. Bennett, "Building and querying an enterprise knowledge graph," *IEEE Transactions on Services Computing*, vol. 12, no. 3, pp. 356–369, 2019.

[2] T. Steiner, R. Verborgh, R. Troncy, J. Gabarro, and D. Van, "Adding realtime coverage to the google knowledge graph," in *Int. Semant. Web Conf.*, 2012, pp. 65–68.

[3] S. Leach-Murray, "The Linked Open Data Cloud," *Tech.Ser.Quar.*, vol. 38, no. 2, pp. 193–194, 2021.

[4] A. Bonifati, W. Martens, and T. Timm, "An analytical study of large SPARQL query logs," *VLDB. J.*, vol. 29, no. 2, pp. 655–679, 2020.

[5] R. Harbi, I. Abdelaziz, P. Kalnis, N. Mamoulis, Y. Ebrahim, and M. Sahli, "Accelerating SPARQL queries by exploiting hash-based locality and adaptive partitioning," *VLDB. J.*, vol. 25, no. 3, pp. 355–380, 2016.

[6] L. Cheng and S. Kotoulas, "Scale-out processing of large rdf datasets," *IEEE Transactions on Big Data*, vol. 1, no. 4, pp. 138 – 150, 2015.

[7] T. Chawla, G. Singh, E. S. Pilli, and M. C. Govil, "Storage, partitioning, indexing and retrieval in Big RDF frameworks: A survey," *Comput. Sci. Rev.*, vol. 38, p. 100309, 2020.

[8] N. Ge, Z. Qin, P. Peng, M. Li, L. Zou, and K. Li, "A cost-driven top-k queries optimization approach on federated rdf systems," *IEEE Transactions on Big Data*, pp. 1–1, 2022.

[9] K. Lee, L. Liu, R. K. Ganti, M. Srivatsa, Q. Zhang, Y. Zhou, and Q. Wang, "Lightweight indexing and querying services for big spatial data," *IEEE Transactions on Services Computing*, vol. 12, no. 3, pp. 343–355, 2019.

[10] S. Gurajada, S. Seufert, I. Miliaraki, and M. Theobald, "TriAD: a distributed shared-nothing RDF engine based on asynchronous message passing," in *Proc. 2014 ACM SIGMOD Int. Conf. Manage data.*, 2014, pp. 289–300.

[11] J. Huang, D. J. Abadi, and K. Ren, "Scalable SPARQL querying of large RDF graphs," *Proc. VLDB Endow.*, vol. 4, no. 11, pp. 1123–1134, 2011.

[12] L. Gai, W. Chen, and T. Wang, "A partition-based summary-graph-driven method for efficient RDF query processing," *arXiv preprint arXiv:1510.07749*, 2015.

[13] A. Madkour, A. M. Aly, and W. G. Aref, "Worq: Workload-driven rdf query processing," in *Int. Semant. Web Conf.*, 2018, pp. 583–599.

[14] A. Madkour, W. G. Aref, and A. M. Aly, "Sparti: Scalable rdf data management using query-centric semantic partitioning," in *Proc. Int. Workshop Semant. Big Data*, 2018, pp. 1–6.

[15] I. Zouaghi, A. Mesmoudi, J. Galicia, L. Bellatreche, and T. Aguili, "GoFast: Graph-based optimization for efficient and scalable query evaluation," *Inform. Syst.*, vol. 99, p. 101738, 2021.

[16] R. Sellami and B. Defude, "Complex queries optimization and evaluation over relational and nosql data stores in cloud environments," *IEEE Transactions on Big Data*, vol. 4, no. 2, pp. 217 – 230, 2018.

[17] J. Song, Y. Bi, G. Han, and T. Li, "FacetsBase: A key-value store optimized for querying on scholarly data," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 1, pp. 302–315, 2018.

[18] N. M. Elzein, M. A. Majid, I. A. T. Hashem, I. Yaqoob, F. A. Alaba, and M. Imran, "Managing big RDF data in clouds: Challenges, opportunities, and solutions," *Sustain. Cities Soc.*, vol. 39, pp. 375–386, 2018.

[19] J. Sun and Q. Jin, "Scalable rdf store based on hbase and mapreduce," in *IEEE 3rd Int. Conf. Adv. Comput. Theory Eng. (ICACTE)*, vol. 1, 2010, pp. V1–633.

[20] D. J. Abadi, A. Marcus, S. R. Madden, and K. Hollenbach, "Scalable semantic web data management using vertical partitioning," in *Proc. 33rd Int. Conf. Very Large Data Bases*, 2007, pp. 411–422.

[21] M. Hassan and S. K. Bansal, "Data partitioning scheme for efficient distributed rdf querying using apache spark," in *IEEE 13th Int. Conf. Semant. Comput. (ICSC)*, 2019, pp. 24–31.

[22] N. Papailiou, I. Konstantinou, D. Tsoumakos, P. Karras, and N. Koziris, "H2RDF+: High-performance distributed joins over large-scale RDF graphs," in *IEEE Int. Conf. Big Data*, 2013, pp. 255–263.

[23] A. Schätzle, M. Przyjaciel-Zablocki, and G. Lausen, "PigSPARQL: Mapping SPARQL to pig latin," in *Proc. Int. Workshop Semant. Web Inform. Manage.*, 2011, pp. 1–8.

[24] A. Schätzle, M. Przyjaciel-Zablocki, S. Skilevic, and G. Lausen, "S2RDF: RDF querying with SPARQL on spark," *Proc. VLDB Endow*, vol. 9, no. 10, pp. 804 –815, 2016.

[25] A. Schätzle, M. Przyjaciel-Zablocki, A. Neu, and G. Lausen, "Sempala: Interactive SPARQL query processing on hadoop," in *Int. Semant. Web Conf.*, 2014, pp. 164–179.

[26] D. Graux, L. Jachiet, P. Genevès, and N. Layaïda, "Sparqlgx: Efficient distributed evaluation of sparql with apache spark," in *Int. Semant. Web Conf.*, 2016, pp. 80–87.

[27] S. Li, D. Shen, Y. Kou, and D. Yang, "Query optimization for massive RDF data based on Spark," in *IEEE 4th Int. Conf. Big Data Comput. Commun. (BIGCOM)*, 2018, pp. 219–224.

[28] M. Hassan and S. K. Bansal, "RDF data storage techniques for efficient SPARQL query processing using distributed computation engines," in *IEEE Int. Conf. Inf. Reuse Integration (IRI)*, 2018, pp. 323–330.

[29] C. Stadler, G. Sejdiu, D. Graux, and J. Lehmann, "Sparklify: A scalable software component for efficient evaluation of sparql queries over distributed rdf datasets," in *Int. Semant. Web Conf.*, 2019, pp. 293–308.

[30] M. Hassan and S. K. Bansal, "S3QLRDF: property table partitioning scheme for distributed SPARQL querying of large-scale RDF data," in *IEEE Int. Conf. Smart. Data Serv. (SMDS)*, 2020, pp. 133–140.

[31] X. Chen, H. Chen, N. Zhang, and S. Zhang, "SparkRDF: elastic discreted RDF graph processing engine with distributed memory," in *IEEE/WIC/ACM Int. Conf. Web Intell. Intell. Agent Technol. (WI-IAT)*, vol. 1, 2015, pp. 292–300.

[32] K. Rohloff and R. E. Schantz, "Clause-iteration with MapReduce to scalably query datagraphs in the SHARD graph-store," in *Proc. 4th Int. Workshop Data-*

Intensive Distri. Comput., 2011, pp. 35–44.

[33] K. Wilkinson and K. Wilkinson, "Jena property table implementation," 2006.

[34] K. Zeng, J. Yang, H. Wang, B. Shao, and Z. Wang, "A distributed graph engine for web scale RDF data," *Proceedings of the VLDB Endowment*, vol. 6, no. 4, pp. 265–276, 2013, publisher: VLDB Endowment.

[35] X. Liu, Y. Tian, Q. He, W.-C. Lee, and J. McPherson, "Distributed graph summarization," in *Proc. 23rd ACM Int. Conf. Conf. Inf. Knowl. Manage.*, 2014, pp. 799–808.

[36] J. Jiang, B. Choi, J. Xu, and S. S. Bhowmick, "A generic ontology framework for indexing keyword search on massive graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 6, pp. 2322–2336, 2019.

[37] P. Peng, Q. Ge, L. Zou, M. T. Özsu, Z. Xu, and D. Zhao, "Optimizing multi-query evaluation in federated RDF systems," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1692–1707, 2019.

[38] M. Bittorf, T. Bobrovytsky, C. Erickson, M. G. D. Hecht, M. Kuff, D. K. A. Leblang, N. Robinson, D. R. S. Rus, J. Wanderman, and M. M. Yoder, "Impala: A modern, open-source sql engine for hadoop," in *Proc. 7th Biennial Conf. Innovative Data Syst. Res.*, 2015.

[39] C. Olston, B. Reed, U. Srivastava, R. Kumar, and A. Tomkins, "Pig latin: a not-so-foreign language for data processing," in *Proc. ACM SIGMOD international conference on Management of data*, 2008, pp. 1099–1110.

[40] F. Goasdoué, Z. Kaoudi, I. Manolescu, J.-A. Quiané-Ruiz, and S. Zampetakis, "CliqueSquare: Flat plans for massively parallel RDF queries," in *IEEE 31st Int. Conf. Data Eng.*, 2015, pp. 771–782.

[41] S. Yang, Z. Yang, X. Chen, J. Zhao, and Y. Ma, "Distributed aggregation-based attributed graph summarization for summary-based approximate attributed graph queries," *Expert Syst.Appl.*, vol. 176, p. 114921, 2021.

[42] W. Ali, M. Saleem, B. Yao, A. Hogan, and A.-C. N. Ngomo, "A survey of rdf stores sparql engines for querying knowledge graphs," *VLDB Journal*, vol. 31, no. 3, pp. 1–26, 2022.

[43] Y. Guo, Z. Pan, and J. Heflin, "Lubm: A benchmark for owl knowledge base systems," *J. Web Semant.*, vol. 3, no. 2-3, pp. 158–182, 2005.

[44] G. Aluç, O. Hartig, M. T. Özsu, and K. Daudjee, "Diversified stress testing of rdf data management systems," in *Int. Semant. Web Conf.*, 2014, pp. 197–212.

[45] I. Abdelaziz, R. Harbi, Z. Khayyat, and P. Kalnis, "A survey and experimental comparison of distributed sparql engines for very large rdf data," *Proc. VLDB Endow.*, vol. 10, no. 13, pp. 2049–2060, 2017.

[46] A. Hogan, E. Blomqvist, M. Cochez, C. dAmato, G. D. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, and A. Zimmermann, "Knowledge graphs," *ACM Comput. Surv.*, vol. 54, no. 4, pp. 71:1–71:37, 2021.

[47] S. T. Mai, S. Amer-Yahia, I. Assent, M. S. Birk, M. S. Dieu, J. Jacobsen, and J. M. Kristensen, "Scalable interactive dynamic graph clustering on multicore cpus," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 7, pp. 1239 – 1252, 2019.

[48] Z. Tang, M. He, Z. Fu, and L. Yang, "Incgraph: An improved distributed incremental graph computing model and framework based on spark graphx," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 6, pp. 2783 – 2797, 2022.

[49] T. Weng, X. Zhou, K. Li, P. Peng, and K. Li, "Efficient distributed approaches to core maintenance on large dynamic graphs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 1, pp. 129 – 143, 2022.

[50] X. Gou, L. Zou, C. Zhao, and T. Yang, "Graph stream sketch: Summarizing graph streams with high speed and accuracy," *IEEE Transactions on Knowledge and Data Engineering*, vol. Early Access, 2022.

[51] Q.-S. Hua, Y. Shi, D. Yu, H. Jin, J. Yu, Z. Cai, X. Cheng, and H. Chen, "Faster parallel core maintenance algorithms in dynamic graphs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 31, no. 6, pp. 1287 – 1300, 2020.